

درس سیستم عامل

تمرین عملی دوم سوال اول

402105727

متین باقری

402105802

فاطمیما تیمارچی

(1

(آ

پس از نصب نرم افزار و زدن دستور gdb-multiarch به خطا خوردم برای همین آدرس نوشته نوشته در خطا را مطابق ویدیو به فایل مورد نظر اضافه کردم . و سپس مطابق تصویر زیر دستور gdb-multiarch به طور صحیح کار کرد :

```
Timarchi-Bagheri@debian:~/Downloads/xv6-riscv-riscv$ gdb-multiarch
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
The target architecture is set to "riscv:rv64".
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0000000080001db6 in ?? ()
(gdb) b syscall
```

مطابق ویدیو دو ترمینال باز کردم و سپس دستورات گفته شده صورت سوال رو اجرا کردم فقط باید به این نکته باید دقت کنیم که پس از دستور زدن c باید در ترمینال دیگه دستور ls رو زدم که gdb به thread جدید switch کنه . و خروجی به این صورت میشود :

```
0x0000000080001db6 in ?? ()
(gdb) b syscall
Breakpoint 1 at 0x8000282a: file kernel/syscall.c, line 136.
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, syscall () at kernel/syscall.c:136
```

سپس دستور layout را زدم و یک پنجره جدید باز شد و سپس دستور backtrace را زدم :

```

--kernel/syscall.c
118 SYS_chdir sys_chdir
119 SYS_dup sys_dup
120 SYS_getpid sys_getpid
121 SYS_sbrk sys_sbrk
122 SYS_pause sys_pause
123 SYS_optime sys_optime
124 SYS_open sys_open
125 SYS_write sys_write
126 SYS_mknod sys_mknod
127 SYS_unlink sys_unlink
128 SYS_link sys_link
129 SYS_mkdir sys_mkdir
130 SYS_close sys_close
131 SYS_top sys_top
132 }
133
134 void
135 syscall(void)
136 {
137     int num;
138     struct proc *p = myproc();
139
140     num = p->trapframe->a7;
141     if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
142         // Use num to lookup the system call function for num, call it,
143         // and store its return value in p->trapframe->a0
144         p->trapframe->a0 = syscalls[num]();
145     } else {
146         printf("bad syscall: unknown sys call %d\n",
147             num);
148         p->trapframe->a0 = -1;
149     }
150 }

```

remote Thread 1.1 (src) In: syscall L136 PC: 0x0000202a

```

(gdb) backtrace
#0  syscall () at kernel/syscall.c:136
#1  0x00000000000025f0 in usertrap () at kernel/trap.c:68
#2  0x0000003fffff09c in ?? ()
(gdb)

```

سوال 1

1- مطابق شکل زیر با زدن دستور backtrace روی خط `proc *p = myproc();` متوقف میشیم و خروجی دستور در عکس زیر هست مطابق خط دو نتیجه میگیریم که user یک syscall زده . از خط اول نتیجه میگیریم که این syscall توسط تابع `usertrap()` وارد حالت کرنل شده (در خط 38 در فایل `usertrap.c` نوشته شده syscall) از خط اول هم نتیجه میگیریم که این syscall داره syscall ای رو فرامیخونه شمارش در رجیستر a7 هست رو اجرا میکنه . (در فایل `syscall.c` در خط 136 نوشته شده `num=r->trapframe->a7`)

```

--kernel/syscall.c
118 SYS_chdir sys_chdir
119 SYS_dup sys_dup
120 SYS_getpid sys_getpid
121 SYS_sbrk sys_sbrk
122 SYS_pause sys_pause
123 SYS_optime sys_optime
124 SYS_open sys_open
125 SYS_write sys_write
126 SYS_mknod sys_mknod
127 SYS_unlink sys_unlink
128 SYS_link sys_link
129 SYS_mkdir sys_mkdir
130 SYS_close sys_close
131 SYS_top sys_top
132 }
133
134 void
135 syscall(void)
136 {
137     int num;
138     struct proc *p = myproc();
139
140     num = p->trapframe->a7;
141     if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
142         // Use num to lookup the system call function for num, call it,
143         // and store its return value in p->trapframe->a0
144         p->trapframe->a0 = syscalls[num]();
145     } else {
146         printf("bad syscall: unknown sys call %d\n",
147             num);
148         p->trapframe->a0 = -1;
149     }
150 }

```

remote Thread 1.1 (src) In: syscall L136 PC: 0x0000202a

```

(gdb) backtrace
#0  syscall () at kernel/syscall.c:136
#1  0x00000000000025f0 in usertrap () at kernel/trap.c:68
#2  0x0000003fffff09c in ?? ()
(gdb)

```

2- حالا برای اینکه بدونیم دقیق کدوم تابع هست دستور `p->trapframe->a7`; رو میزنیم تا بدونیم دقیقا کدوم تابع فراخوانی شده. مطابق عکس زیر مقدار `a7` برابر `f` که همان 15 است پس مطابق فایل `kernel/syscall.h` این شماره متعلق به `SYS_open` هست پس اسم تابع `SYS_open` است پس `syscall`, `open` را صدا زده.

```
(gdb) p/x p->trapframe->a7
$2 = 0xf
(gdb)
```

```
C: syscall.h
1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_fstat 8
10 #define SYS_chdir 9
11 #define SYS_dup 10
12 #define SYS_getpid 11
13 #define SYS_sbrk 12
14 #define SYS_pause 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
```

3- حالا مطابق با زدن دستور `n` مطابق عکس زیر، از خط `proc *p = myproc();` عبور کردم:

```
kernel/syscall.c
118 SYS_chdir sys_chdir
119 SYS_dup sys_dup
120 SYS_getpid sys_getpid
121 SYS_sbrk sys_sbrk
122 SYS_pause sys_pause
123 SYS_uptime sys_uptime
124 SYS_open sys_open
125 SYS_write sys_write
126 SYS_mknod sys_mknod
127 SYS_unlink sys_unlink
128 SYS_link sys_link
129 SYS_mkdir sys_mkdir
130 SYS_close sys_close
131 SYS_top sys_top
132
133
134 void
135 syscall(void)
136 {
137     int num;
138     struct proc *p = myproc();
139
140     num = p->trapframe->a7;
141     if (num < 0 || num > NITEM(syscalls) || num > syscalls[num]) {
142         // Use num to lookup the system call function for num, call it,
143         // and store its return value in p->trapframe->a0
144         p->trapframe->a0 = syscalls[num];
145     } else {
146         printf("unknown sys call %d\n",
147             p->pid, p->name, num);
148         p->trapframe->a0 = -1;
149     }
150 }

Kernel Thread 1.1 (src) in: syscall
(gdb) backtrace
#0  syscall () at kernel/syscall.c:136
#1  0x00000000000021fa in usertrap () at kernel/trap.c:68
#2  0x0000000000000000 in ?? ()
(gdb) n
(gdb) n
(gdb)
```

4- سپس دستور $p/x * p$ را اجرا کردم و پرده قلعی را در مبنای 16 مشاهده کردم :

```

118 SVS_chdir sys_chdir
119 SVS_dup sys_dup
120 SVS_getpid sys_getpid
121 SVS_brk sys_brk
122 SVS_pause sys_pause
123 SVS_uptime sys_uptime
124 SVS_open sys_open
125 SVS_write sys_write
126 SVS_mknod sys_mknod
127 SVS_unlink sys_unlink
128 SVS_link sys_link
129 SVS_mkdir sys_mkdir
130 SVS_close sys_close
131 SVS_top sys_top
132 }
133
134 void
135 syscall(void)
136 {
137     int num;
138     struct trap * p = myproc();
139
140     num = p->trapframe->a7;
141     if (num < 0 || num > N_SYSCALL)
142         // use num to choose the system call function for num, call it,
143         // and store its return value in p->trapframe->a0
144         p->trapframe->a0 = syscall(num);
145     else
146         printf("bad system call %d\n", num);
147     p->pid = p->name;
148     p->trapframe->a0 = 0;
149 }
150
(gdb) backtrace
#0  syscall () at kernel/syscall.c:136
#1  0x00000000000025f8 in usertrap () at kernel/trap.c:60
#2  0x0000000000000000 in ?? ()
(gdb) n
(gdb) n
(gdb) p/x *p
1 = {locked = 0x0, name = 0x00007170, cpu = 0x0, state = 0x4, chan = 0x0, killed = 0x0,
  xstate = 0x0, pid = 0x2, parent = 0x000127c8, kstack = 0x3ffff000, ui = 0x5000, compatable = 0x7f61000,
  trapframe = 0x87f55000, context = {fs = 0x00001c22, is = 0x3ffff000, n = 0x3ffff000, si = 0x00012030,
  di = 0x00012030, si = 0x1, si = 0x40ff, si = 0x1, si = 0x1, si = 0x0, si = 0x0, si = 0x2020, si = 0x0,
  si = 0x0000000000000000}, d1 = {0x000224c0, 0x000224c0, 0x000224c0, 0x000224c0, 0x000224c0, 0x000224c0,
  0x000224c0, 0x000224c0}, name = {0x73, 0x60, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0},
  uid = 0x00020000, num = 0x0}
(gdb)

```

سوال 2

1-دستور $p->trapframe->a7$ رو مطابق عکس زیر زدم و خروجیش برابر $0xf = 2\$$ شد . این مقدار شماره syscall درخواست شده توسط اولین برنامه کاربر (initcode) رو نشون میده . بنابراین در اینجا برنامه init اولین بار syscall شماره 15 رو فراخوانی کرده . برای اینکه بفهمیم کدوم تابع فراخوانی شده باید فایل `kernel/syscall.h` را ببینیم . مطابق شکل نوم عدد 5 مربوط به `SYS_open` هست بنابراین syscall مربوط به تابع `SYS_open` هست

```

(gdb) p/x p->trapframe->a7
$2 = 0xf
(gdb)

```

```

1 // System call numbers
2 #define SVS_fork 1
3 #define SVS_exit 2
4 #define SVS_wait 3
5 #define SVS_pipe 4
6 #define SVS_read 5
7 #define SVS_kill 6
8 #define SVS_exec 7
9 #define SVS_fstat 8
10 #define SVS_chdir 9
11 #define SVS_dup 10
12 #define SVS_getpid 11
13 #define SVS_brk 12
14 #define SVS_pause 13
15 #define SVS_uptime 14
16 #define SVS_open 15
17 #define SVS_write 16
18 #define SVS_mknod 17
19 #define SVS_unlink 18
20 #define SVS_link 19
21 #define SVS_mkdir 20
22 #define SVS_close 21

```

2- سپس مطابق صورت سوال دستور `p/x $sstatus` رو در ترمینال زدم و خروجی آن در عکس زیر موجود است . خروجی این دستور نشون میده که مقدار درون رجیستر `sstatus` چی هست . (رجیستر `sstatus` وضعیت پردازنده فعلی رو نشون میده) :

```
(gdb) p/x $sstatus
s3 = 0x200000022
(gdb)
```

سوال 3

بعد از زدن دستور `p/x $sstatus` مطابق عکس بالا عدد `0x200000022` نشون داده میشه و این نشون دهنده وضعیت پردازنده هست برای فهمیدن اینکه حالت قبلی پردازنده چی بود بیت `SSP` رو باید چک کنیم (بیت `SSP` نشون دهنده حالت قبلی پردازنده قبل از `trap` هست . بیت `SSP` هشتم است که در عدد به دست آمده بیت هشتم یا `SSP` برابر 0 است میدونیم اگر بیت ما برابر 0 برای `user mode` است و اگر بیت ما برابر 1 باشد در `kernel mode` هستیم , پس CPU قبل از `trap` در حالت `user mode` بوده است .

(ب)

در این قسمت باید با زدن دستور `top` مقدار `PID` پردازنده ها و دستوری که به واسطه اون اجرا شده و مقدار حافظه اختصاص داده شده رو میدیدیم . که من اینکار رو انجام دادم و ترمینالم به صورت زیر شد :

```
xv6-riscv-riscv: make - Konsole
Timarchi-Bagheri@debian:~$ ls
Desktop Documents Downloads gdb-file Music Pictures Public Templates Videos
Timarchi-Bagheri@debian:~$ cd Downloads
Timarchi-Bagheri@debian:~/Downloads$ ls
ayeh.zip gdb.txt xv6-riscv-riscv xv6-riscv-riscv.zip
Timarchi-Bagheri@debian:~/Downloads$ cd xv6-riscv-riscv
Timarchi-Bagheri@debian:~/Downloads/xv6-riscv-riscv$ ls
fs.img kernel LICENSE Makefile mkfs README test-xv6.py user
Timarchi-Bagheri@debian:~/Downloads/xv6-riscv-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

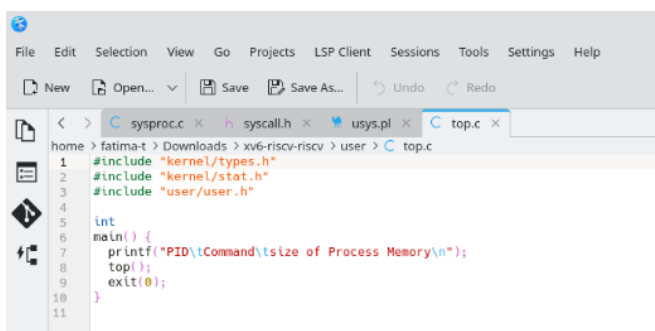
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ top
PID      Command size of Process Memory
1        init    16384
2        sh      20480
3        top     16384
```

حالا مرحله به مرحله فایل هایی که تغییر دادم رو ذکر میکنم (فایل های تغییر داده شده رو در این پوشه قرار دادم) :

1- ساختن فایل top.c :

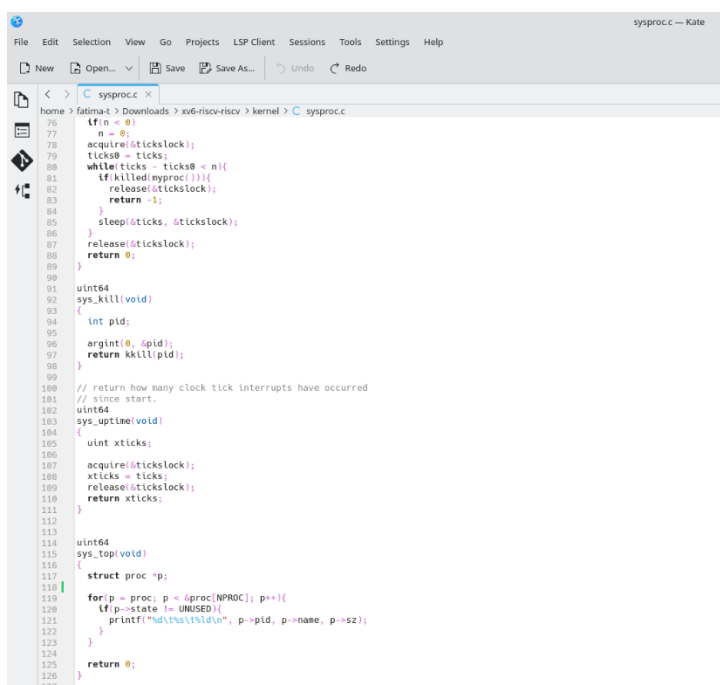
در این فایل من تابع top ای که خورم ساختم رو صدا میکنه .



```
1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 int
6 main() {
7     printf("PID\tCommand\tSize of Process Memory\n");
8     top();
9     exit(0);
10 }
11
```

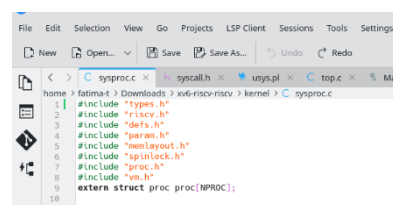
2- تغییر در فایل sysproc.c

ابتدا تابع sys_top را اضافه کردم . تعریف کلی این تابع دقیقا مطابق تملی توابع موجود در این فایل بود و در این تابع من یک حلقه میزنم روی process ها که مقدار PID و اسم دستوری که به واسطه اون اجرا شده و مقدار حافظه اختصاص داده شد رو پرینت میکنم :



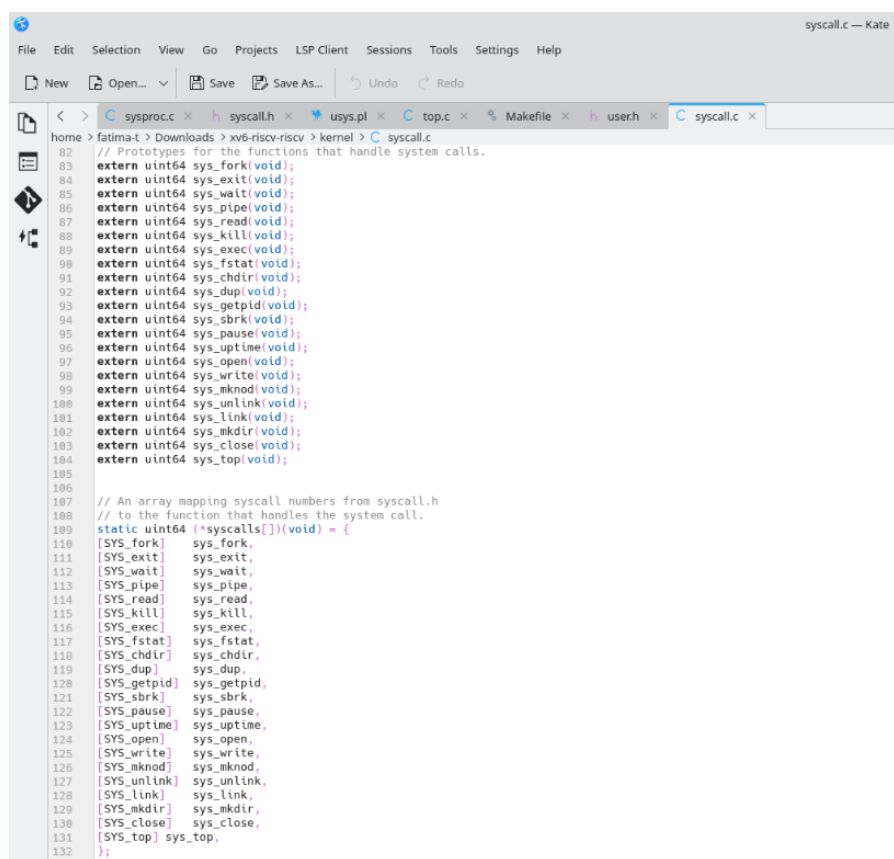
```
76 if(n < 0)
77     n = 0;
78 acquire(&tickslock);
79 ticks0 = ticks;
80 while(ticks - ticks0 < n){
81     if(killed(myproc)){
82         release(&tickslock);
83         return -1;
84     }
85     sleep(&ticks, &tickslock);
86 }
87 release(&tickslock);
88 return 0;
89 }
90
91 uint64
92 sys_kill(void)
93 {
94     int pid;
95     argint(0, &pid);
96     return kkill(pid);
97 }
98
99 // return how many clock tick interrupts have occurred
100 // since start.
101 uint64
102 sys_uptime(void)
103 {
104     uint xticks;
105     acquire(&tickslock);
106     xticks = ticks;
107     release(&tickslock);
108     return xticks;
109 }
110
111
112
113
114 uint64
115 sys_top(void)
116 {
117     struct proc *p;
118     for(p = proc; p < &proc[NPROC]; p++){
119         if(p->state != UNUSED){
120             printf("%d\t%s\t%d\n", p->pid, p->name, p->sz);
121         }
122     }
123     return 0;
124 }
125
126
127
```

یکی از مشکلاتی که من داشتم ارور `proc is not declared` بود برای رفع این مشکل هم این تغییر را در فایل `sysproc.c` انجام اعمال کردم تا در این فایل بتوانم به `proc` دسترسی داشته باشم:



3- تغییر در فایل `syscall.c`

در این فایل دو خط اضافه کردم اولی که دقیقاً مطابق بقیه خطوط باید `uint64` رو `extern` کردم تا بتونه بهش دسترسی داشته باشه. دومین خط هم برای این اضافه کردم که وقتی ما دستور `top` رو میزنیم سیستم عامل طبق این خط میدونه که باید تابع `SYS_top` رو اجرا کنه:



4- تغییر در فایل `syscall.h`

در این فایل اعداد برای این هستند که وقتی ما یک دستور رو میزنیم kernel میدونه مطابق این اعداد باید کدوم syscall رو انجام بده پس برای تابع SYS_top هم من یک عدد بهش اختصاص دادم که این عدد برابر اعداد بقیه syscall ها نباشه :

```

1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_chdir 8
10 #define SYS_dup 9
11 #define SYS_getpid 10
12 #define SYS_stat 11
13 #define SYS_chmod 12
14 #define SYS_pause 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_top 22
24

```

5- تغییر در فایل user.h :

تابع جدید رو که top بود رو توسط int top(void) به این فایل اضافه کردم :

```

1 #define SBRK_ERROR ((char *)-1)
2
3 struct stat;
4
5 // system calls
6 int fork(void);
7 int exit(int) __attribute__((noreturn));
8 int wait(int*);
9 int pipe(int*);
10 int write(int, const void*, int);
11 int read(int, void*, int);
12 int close(int);
13 int kill(int);
14 int exec(const char*, char**);
15 int open(const char*, int);
16 int mknod(const char*, short, short);
17 int unlink(const char*);
18 int fstat(int fd, struct stat*);
19 int link(const char*, const char*);
20 int mkdir(const char*);
21 int chdir(const char*);
22 int dup(int);
23 int getpid(void);
24 char* sys_sbrk(int, int);
25 int pause(int);
26 int uptime(void);
27
28 // ulib.c
29 int stat(const char*, struct stat*);
30 char* strcpy(char*, const char*);
31 void* memmove(void*, const void*, int);
32 char* strchr(const char*, char c);
33 int strcmp(const char*, const char*);
34 char* gets(char*, int max);
35 uint strlen(const char*);
36 void* memset(void*, int, uint);
37 int atoi(const char*);
38 int memcmp(const void*, const void*, uint);
39 void* memcpy(void*, const void*, uint);
40 char* sbrk(int);
41 char* sbrkleazy(int);
42
43 // printf.c
44 void fprintf(int, const char*, ...) __attribute__((format(printf, 2, 3)));
45 void printf(const char*, ...) __attribute__((format(printf, 1, 2)));
46
47 // umalloc.c
48 void* malloc(uint);
49 void free(void*);
50
51 int top(void);
52

```

6- تغییر در فایل usys.pl :

خط ("top") entry رو اضافه کردم برای اینکه کاربر بتونه با زدن دستورات مخصوص بتونه شماره syscall ای که میخواد رو فراخوانی کنه و اون دستور رو اجرا کنه :

```

1 #!/usr/bin/perl -w
2
3 # Generate usys.S, the stubs for syscalls.
4
5 print "# generated by usys.pl - do not edit!\n";
6
7 print "#include <kernel/syscall.h>\n";
8
9 sub entry {
10     my $prefix = "sys_";
11     my $name = shift;
12     if ($name eq "brk") {
13         print "global $prefix$name\n";
14         print "$prefix$name:\n";
15     } else {
16         print "global $name\n";
17         print "$name:\n";
18     }
19     print "    ll a7, SYS_$name\n";
20     print "    ecall\n";
21     print "    ret\n";
22 }
23
24 entry("fork");
25 entry("exit");
26 entry("wait");
27 entry("pipe");
28 entry("read");
29 entry("write");
30 entry("close");
31 entry("kill");
32 entry("exec");
33 entry("open");
34 entry("mknode");
35 entry("unlink");
36 entry("fstat");
37 entry("link");
38 entry("mkdir");
39 entry("chdir");
40 entry("dup");
41 entry("getpid");
42 entry("brk");
43 entry("pause");
44 entry("uptime");
45 entry("top");
46

```

7- تغییر در فایل makefile

در انتها که تمام کد هارو نوشتم باید در فایل makefile دستور top رو به صورت زیر اضافه کردم :

```

100 %: %.o $(ULIB) $@/user.ld
101     $(LD) $(LDFLAGS) -T $@/user.ld -o $@ $(ULIB)
102     $(OBJDUMP) -S $@ > $*.asm
103     $(OBJDUMP) -t $@ | sed 's/,SYMBOL TABLE/d; s/ .* / /; /"$$/d' > $*.sym
104
105 $U/usys.S : $U/usys.pl
106     perl $U/usys.pl > $U/usys.S
107
108 $U/usys.o : $U/usys.S
109     $(CC) $(CFLAGS) -c -o $U/usys.o $U/usys.S
110
111 $U/_forktest: $U/forktest.o $(ULIB)
112     # forktest has less library code linked in - needs to be small
113     # in order to be able to max out the proc table.
114     $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/forktest.o $U/ulib.o $U/usys.o
115     $(OBJDUMP) -S $U/_forktest > $U/forktest.asm
116
117 mkfs/mkfs: mkfs/mkfs.c $K/fs.h $K/param.h
118     gcc -Wno-unknown-attributes -I. -o mkfs/mkfs mkfs/mkfs.c
119
120 # Prevent deletion of intermediate files, e.g. cat.o, after first build, so
121 # that disk image changes after first build are persistent until clean. More
122 # details:
123 # http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
124 .PRECIOUS: %.o
125
126 UPROGS=\
127     $U/_cat\
128     $U/_echo\
129     $U/_forktest\
130     $U/_grep\
131     $U/_lnt\
132     $U/_kill\
133     $U/_ln\
134     $U/_ls\
135     $U/_mdu\
136     $U/_rm\
137     $U/_sh\
138     $U/_stressfs\
139     $U/_usertests\
140     $U/_grind\
141     $U/_wcv\
142     $U/_zombie\
143     $U/_logstress\
144     $U/_forphan\
145     $U/_dorphan\
146     $U/_top\
147
148 fs.ing: mkfs/mkfs README $(UPROGS)
149     mkfs/mkfs fs.ing README $(UPROGS)
150
151 -include kernel/*.d user/*.d
152

```

عملکرد کلی : پس حالا وقتی ما دستور `make qemu` رو میزنیم با استفاده از فایل `makefile` همه توابع نوشته شده در این فایل به صورت فایل باینری ساخته میشن از جمله `top` . سپس با زدن دستور `top` از حالت `user` به حالت `kernel` میریم و برنامه `top.c` اجرا میشه و در این فایل تابع `top()` صدا زده میشه پس `syscall` زده میشه و سپس ورودی `syscall` شماره `SYS_top` رو پیدا میکنه و سپس کرنل `sys_top` رو اجرا میکنه و سپس به آرایه پردازنده های کرنل دسترسی پیدا میکنیم و اون ها رو در ترمینال چاپ میکنه و پس از اتمام حلقه تابع `0` را `return` میکنه و `exit` میشه بنابراین از حالت کرنل به `user` برمیگردیم .